
1 INFORMATIONS GÉNÉRALES

Élève :	Nom: _____ Prénom: _____
Lieu de travail :	ETML / Avenue de Valmont 28b, 1010 Lausanne
Client :	Nom: ETML Prénom: ETML
Dates de réalisation :	Trimestre 1 ou 3
Temps total :	24 ou 32 périodes

2 PROCÉDURE

- Tous les apprentis réalisent le projet sur la base d'un cahier des charges.
- Le cahier des charges est présenté, commenté et discuté en classe.
- Les apprentis sont entièrement responsables de la sécurité et sauvegarde de leurs données.
- En cas de problèmes graves, les apprentis avertissent le client au plus vite.
- Les apprentis ont la possibilité d'obtenir de l'aide externe, mais ils doivent le mentionner.
- Les informations utiles à l'évaluation de ce projet sont disponibles au chapitre 10

3 TITRE

OO them all

4 SUJET

Réaliser 3 versions d'une application parmi les suggestions suivantes :

4.1 Shoot'em Up Vertical (Style Space Invaders)

- **Entités** : Vaisseau (joueur), Aliens (ennemis), Missiles, Obstacles
- **Fonctionnalités V1** : DéplacerJoueur(), TirerMissile(), DéplacerEnnemi(), GérerCollisions()
- **Fonctionnalités V2** : GameLogic.Update(), Display.RenderGame(), InputHandler.ProcessInput()
- **Fonctionnalités V3** : Player.Move(), Enemy.Update(), Projectile.CheckCollision(), Game.Run()

4.2 Shoot'em Up Horizontal (Style R-Type)

- **Entités** : Chasseur (joueur), Vagues d'ennemis, Projectiles, Décor défilant
- **Fonctionnalités V1** : `DeplacerChasseur()`, `GererDefilement()`, `CreerVagueEnnemis()`
- **Fonctionnalités V2** : `ScrollingManager.Update()`, `WaveManager.SpawnEnemies()`
- **Fonctionnalités V3** : `Fighter.Shoot()`, `EnemyWave.Advance()`, `Background.Scroll()`

4.3 Tower Defense Simplifié

- **Entités** : Tours de défense, Ennemis mobiles, Chemin, Projectiles
- **Fonctionnalités V1** : `PlacerTour()`, `DeplacerEnnemis()`, `TirerTour()`, `CalculerDegats()`
- **Fonctionnalités V2** : `TowerManager.HandleTargeting()`, `PathManager.MoveEnemies()`
- **Fonctionnalités V3** : `Tower.AcquireTarget()`, `Enemy.FollowPath()`, `Projectile.Track()`

4.4 Simulation Bancomat

- **Entités** : Bancomat, Compte, Transaction, Carte bancaire
- **Fonctionnalités V1** : `VerifierCode()`, `RetirerArgent()`, `ConsulterSolde()`, `ImprimerRecu()`
- **Fonctionnalités V2** : `AuthenticationManager.ValidateCard()`, `TransactionProcessor.WithdrawCash()`
- **Fonctionnalités V3** : `ATM.ProcessTransaction()`, `Account.Withdraw()`, `Transaction.Log()`

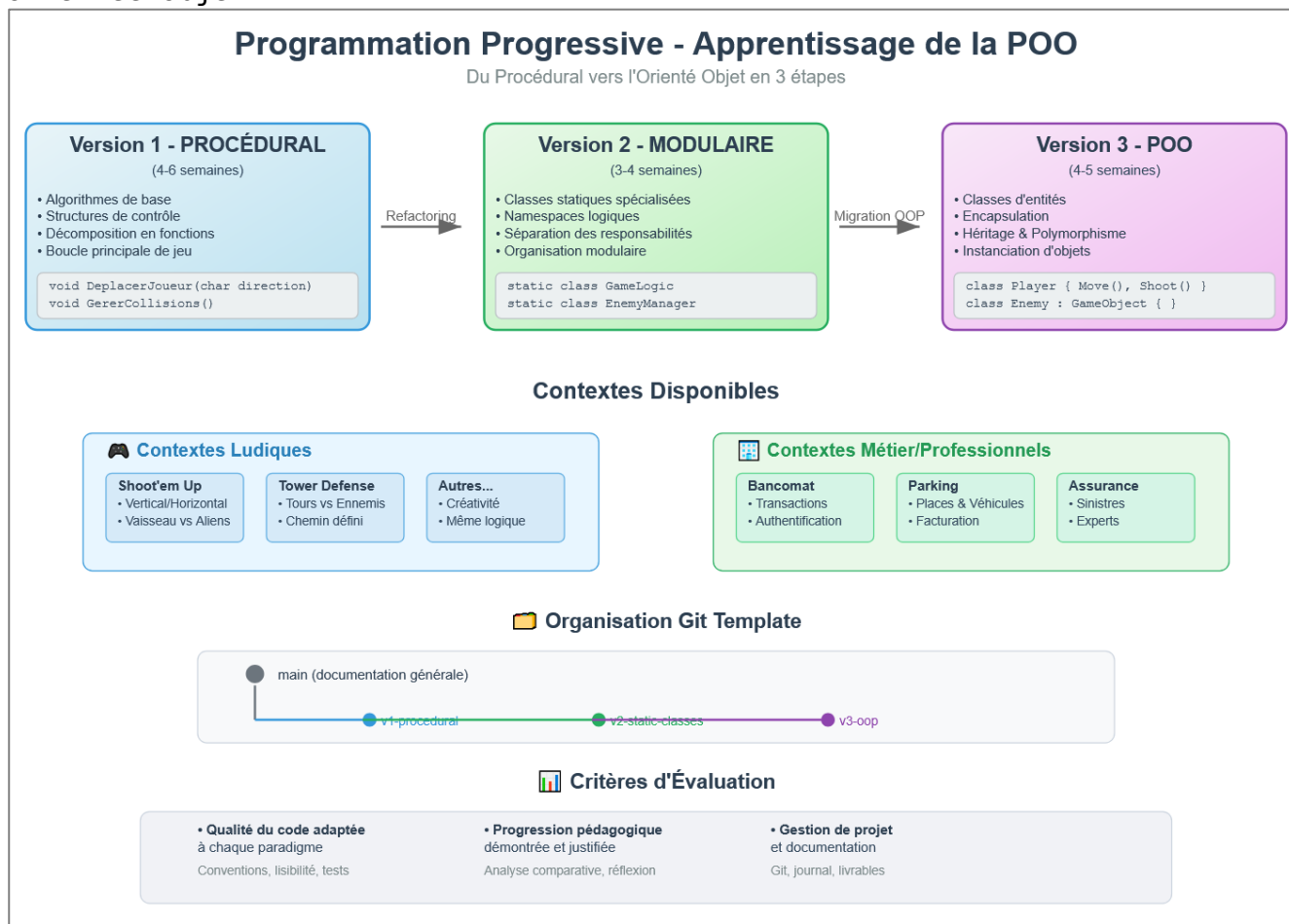
4.5 Gestion de Parking

- **Entités** : Parking, Place, Véhicule, Ticket, Barrière
- **Fonctionnalités V1** : `EntrerVehicule()`, `TrouverPlace()`, `CalculerTarif()`, `SortirVehicule()`
- **Fonctionnalités V2** : `ParkingManager.FindSpot()`, `BillingSystem.CalculateFee()`
- **Fonctionnalités V3** : `Vehicle.Park()`, `ParkingSpot.Occupy()`, `Ticket.CalculateDuration()`

4.6 Système de Réservation d'Assurance

- **Entités** : Client, Police d'assurance, Sinistre, Expert, Dossier
- **Fonctionnalités V1** : `DeclarerSinistre()`, `AssignerExpert()`, `CalculerIndemnité()`, `CloturerDossier()`
- **Fonctionnalités V2** : `ClaimManager.ProcessClaim()`, `ExpertAssignment.FindAvailable()`
- **Fonctionnalités V3** : `Client.FileClaim()`, `Insurance.CalculateCoverage()`, `Expert.Assess()`

Ce projet suit une démarche d'apprentissage progressive en trois étapes distinctes, permettant de maîtriser progressivement la programmation orientée objet :



4.7 Version 1 - Approche Procédurale Console (2 semaines)

- **Objectif** : Maîtriser les algorithmes de base et structures de contrôle
- **Organisation** : Toutes les fonctions dans Program.cs
- **Focus** : Logique de jeu, boucles, tableaux, conditions
- **Livrable** : Jeu fonctionnel en mode console

4.8 Version 2 - Modularité avec Classes Statiques (2 semaines)

- **Objectif** : Apprendre l'organisation modulaire du code
- **Organisation** : Classes statiques spécialisées (GameLogic, Display, InputHandler)
- **Focus** : Namespaces, séparation des responsabilités, refactoring
- **Livrable** : Code restructuré, fonctionnalités identiques

4.9 Version 3 - Programmation Orientée Objet Complète (2 semaines)

- **Objectif** : Maîtriser l'instanciation et l'encapsulation
- **Organisation** : Classes d'entités (Player, Enemy, Projectile, Game)
- **Focus** : Instanciation, encapsulation, héritage, polymorphisme
- **Livrable** : Application orientée objet avec interface graphique (optionnel)

5 MATÉRIEL ET LOGICIEL À DISPOSITION

- Un PC ETML
- Accès à Internet

6 PRÉREQUIS

- Modules de programmation de base
- ICT-320 en cours

7 CAHIER DES CHARGES

Selon le choix du domaine (voir démos fournies), par exemple pour la gestion du parking :

1. Entrée d'un véhicule
 - a. Par numéro d'immatriculation
 - b. Vérifier si une place est encore libre
 - c. Réserver la place
 - d. Enregistrer l'heure
2. Sortie d'un véhicule
 - a. Par numéro d'immatriculation
 - b. Calculer le prix
 - c. Libérer la place
3. Afficher l'état du parking
 - a. Version graphique console :

```
Plan du parking (L=Libre, X=Occupé):
|01:X| |02:L| |03:L| |04:L| |05:L|
|06:X| |07:L| |08:L| |09:L| |10:L|
|11:L| |12:L| |13:L| |14:L| |15:L|
|16:L| |17:L| |18:L| |19:L| |20:L|
```

4. Rechercher un véhicule
5. Statistiques du jour
6. Historique des transactions

8 EXIGENCES TECHNIQUES PAR VERSION

8.1 Version 1 - Exigences Procédurales

- Minimum 5 fonctions distinctes avec paramètres
- Utilisation de tableaux pour gérer les entités
- Interface **console** claire et ergonomique

8.2 Version 2 - Exigences Modulaires

- Minimum 3 classes statiques spécialisées
- Organisation en namespaces logiques
- Séparation claire des responsabilités
- Aucune duplication de code
- Documentation des classes et méthodes

8.3 Version 3 - Exigences Orientées Objet

- Minimum 4 classes d'entités avec instanciation
- Encapsulation correcte (propriétés privées, méthodes publiques)
- Au moins une relation d'héritage
- Constructeurs avec paramètres
- Tests unitaires pour les classes principales

9 LIVRABLES ET PLANNING

Date	Version	Livrables
Semaine 1	Setup	URL du dépôt contenant un dossier « doc » avec le journal de travail et le rapport (introduction/choix du domaine)
Semaine 3	V1 - Procédural	Release v1
Semaine 5	V2 - Modulaire	Release v2
Semaine 8	V3 - P00	Release v3

10 CRITÈRES D'ÉVALUATION

10.1 Connaissances professionnelles / techniques enseignées

- Organisation et structure du code adaptée à chaque version
- Respect des conventions ETML
 - Absence de duplication (DRY - Don't Repeat Yourself)
 - Nommage explicite des variables, méthodes et classes
- Tests unitaires (version 3)
- Interdiction d'utiliser du code généré par IA

10.2 Processus de travail

- Journal de travail détaillé et régulier
- Utilisation correcte de Git avec branches par version
- Proactivité et communication avec le client
- Rapport minimal contenant
 - Introduction
 - Objectifs pédagogiques (ce qui va être appris)
 - Choix du contexte
 - Analyse comparative entre les versions
 - Identification des avantages/inconvénients de chaque approche
 - Réflexion sur l'évolution de la conception
 - Conclusion

10.3 Rythme de travail

- Respect des délais de livraison

11 ORGANISATION du code

11.1 Structure des Branches

Utilisez les branches suivantes :

- **v1-procedural** : Code procédural complet
- **v2-static-classes** : Refactoring avec classes statiques
- **v3-oo** : Version orientée objet finale

11.2 Organisation du dépôt

Le dépôt contient au moins les éléments suivants :

- **Dossier src/** : Code source
- **Dossier doc/** : Documentation
- **README.md** : Description du projet

11.3 Messages de commits

Utilisez des messages de commit explicites, par exemple :

- feat(gameplay) : ajout du tir ennemi
- fix(retrait) : mise à jour du solde du compte client immédiate

12 RESSOURCES RECOMMANDÉES

12.1 Documentation Technique

- Microsoft C# Documentation - Types et classes
- "Clean Code" de Robert C. Martin
- "Refactoring" de Martin Fowler - Patterns de migration
- Support de cours ICT-320